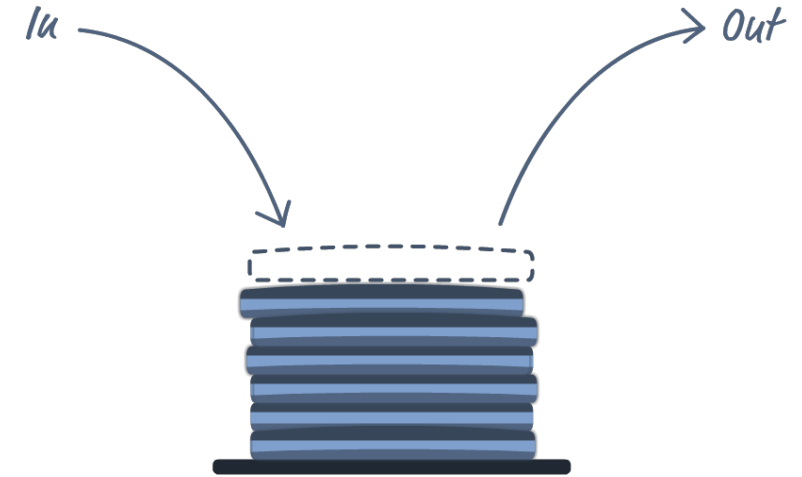
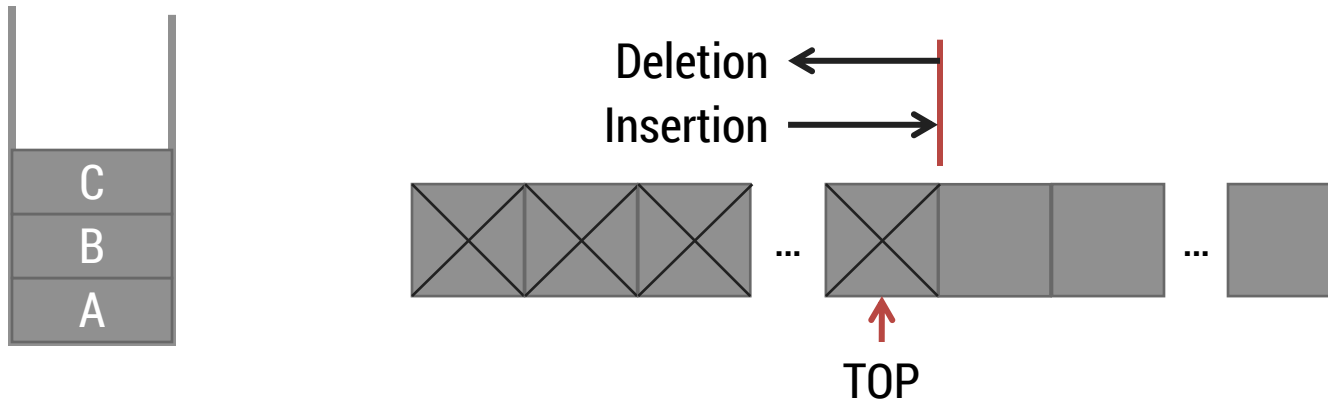


Linear Data Structure Stack



Stack

- ▶ A linear list which allows insertion and deletion of an element at one end only is called **stack**.
- ▶ The insertion operation is called as **PUSH** and deletion operation as **POP**.
- ▶ The most accessible elements in stack is known as **top**.
- ▶ The elements can only be removed in the opposite orders from that in which they were added to the stack.
- ▶ Such a linear list is referred to as a **LIFO (Last In First Out)** list.



Stack Cont...

- ▶ A pointer TOP keeps track of the top element in the stack.
- ▶ Initially, when the *stack is empty*, **TOP** has a value of *“zero”*.
- ▶ Each time a **new element is inserted** in the stack, the pointer is *incremented by “one”* before, the element is placed on the stack.
- ▶ The pointer is *decremented by “one”* each time a **deletion** is made from the stack.

Applications of Stack

- ▶ Recursion
- ▶ Keeping track of function calls
- ▶ Evaluation of expressions
- ▶ Reversing characters
- ▶ Servicing hardware interrupts
- ▶ Solving combinatorial problems using backtracking
- ▶ Expression Conversion (Infix to Postfix, Infix to Prefix)
- ▶ Game Playing (Chess)
- ▶ Microsoft Word (Undo / Redo)
- ▶ Compiler – Parsing syntax & expression
- ▶ Finding paths

Procedure : PUSH (S, TOP, X)

- ▶ This procedure inserts an element **X** to the top of a stack.
- ▶ Stack is represented by a array **S** containing **N** elements.
- ▶ A pointer **TOP** represents the top element in the stack.

1. [Check for stack overflow]

If $TOP \geq N$

Then write ('STACK OVERFLOW')

Return

2. [Increment TOP]

$TOP \leftarrow TOP + 1$

3. [Insert Element]

$S[TOP] \leftarrow X$

4. [Finished]

Return

Stack is empty, $TOP = 0, N=3$

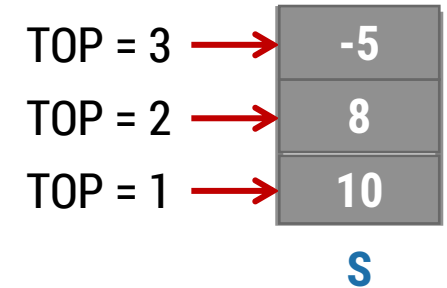
PUSH(S, TOP, 10)

PUSH(S, TOP, 8)

PUSH(S, TOP, -5)

PUSH(S, TOP, 6)

Overflow



*Assuming indexing starts from 1

Function : POP (S, TOP)

- ▶ This function **removes & returns** the top element from a stack.
- ▶ Stack is represented by a array **S** containing **N** elements.
- ▶ A pointer **TOP** represents the top element in the stack.

1. [Check for stack underflow]

```
If      TOP = 0
Then  write ('STACK UNDERFLOW')
      Return (0)
```

2. [Decrement TOP]

```
TOP ← TOP - 1
```

3. [Return former top element of stack]

```
Return(S[TOP + 1])
```

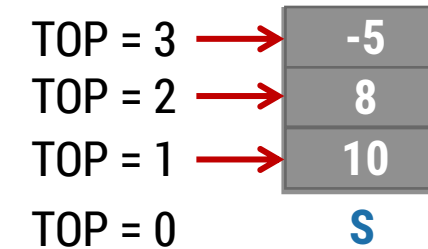
POP(S, TOP)

POP(S, TOP)

POP(S, TOP)

POP(S, TOP)

Underflow



*Assuming indexing starts from 1

Function : PEEP (S, TOP, I)

- ▶ This function returns the value of the **Ith** element from the **TOP** of the stack. The element is not deleted by this function.
- ▶ Stack is represented by a array **S** containing **N** elements.

1. [Check for stack underflow]

If $TOP - I + 1 \leq 0$

Then write ('STACK UNDERFLOW')

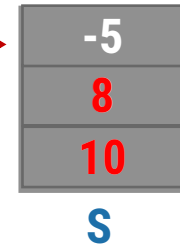
Return (0)

2. [Return Ith element from top of the stack]

Return($S[TOP - I + 1]$)

PEEP (S, TOP, 2)

TOP = 3



PEEP (S, TOP, 3)

PEEP (S, TOP, 4)

Underflow

*Assuming indexing starts from 1

PROCEDURE : CHANGE (S, TOP, X, I)

- ▶ This procedure changes the value of the I^{th} element from the top of the stack to X .
- ▶ Stack is represented by a array S containing N elements.

1. [Check for stack underflow]

If $TOP - I + 1 \leq 0$

Then write ('STACK UNDERFLOW')

Return

2. [Change I^{th} element from top of the stack]

$S[TOP - I + 1] \leftarrow X$

3. [Finished]

Return

CHANGE (S, TOP, 50, 2) TOP = 3 →

CHANGE (S, TOP, 9, 3)

CHANGE (S, TOP, 25, 8)

Underflow

-5
50
9
S

*Assuming indexing starts from 1

Algorithm: RECOGNIZE

Write an algorithm which will check that the given string belongs to following grammar or not.

$L = \{wcw^R \mid w \in \{a,b\}^*\}$ (Where w^R is the reverse of w)

Example of valid strings : **abcba** **aabbcbbaa**

Example of Invalid strings: **aabcaab**

- Given an input string named **STRING** on the alphabet **{a, b, c}** which contains a blank in its rightmost character position.
- Function **NEXTCHAR** returns the next symbol in STRING.
- This algorithm determines whether the contents of STRING belong to the above language.
- The vector **S** represents the stack and **TOP** is a pointer to the top element of the stack.

Algorithm: RECOGNIZE

1. [Initialize stack by placing a letter 'c' on the top]

TOP $\leftarrow$ 1

S [TOP] $\leftarrow$ 'c'

2. [Get and PUSH symbols until either 'c']

NEXT $\leftarrow$ NEXTCHAR (STRING)

Repeat while NEXT $\neq$ 'c'

 Call PUSH (S, TOP, NEXT)

 NEXT $\leftarrow$ NEXTCHAR (STRING)

3. [Scan characters following 'c'; Compare them to the characters on stack]

valid=1

Repeat While S [TOP] $\neq$ 'c'

 NEXT $\leftarrow$ NEXTCHAR (STRING)

 X $\leftarrow$ POP (S, TOP)

 If NEXT $\neq$ X

 Then valid=0;

 Exit

4. [Next symbol must be blank]

If valid = 1

Then Write ('VALID STRING')

Else Write ('INVALID STRING')

Exit

Algorithm: RECOGNIZE

1. [Initialize stack by placing a letter 'c' on the top]

$TOP \leftarrow 1$

$S[TOP] \leftarrow 'c'$

2. [Get and PUSH symbols until either 'c' or blank is encountered]

$NEXT \leftarrow NEXTCHAR (STRING)$

Repeat while $NEXT \neq 'c'$

If $NEXT = ' '$

Then Write ('Invalid String')
Exit

Else Call PUSH (S, TOP, NEXT)
 $NEXT \leftarrow NEXTCHAR (STRING)$

Input String: a b c b a □

↑
NEXT

	Character Scanned	Stack Content
1 {		c
	a	ca
2 {	b	cab
	c	cab

Algorithm: RECOGNIZE

3. Scan characters following 'c';
Compare them to the characters on stack

Repeat While $S[\text{TOP}] \neq 'c'$

$\text{NEXT} \leftarrow \text{NEXTCHAR}(\text{STRING})$

$X \leftarrow \text{POP}(S, \text{TOP})$

 If $\text{NEXT} \neq X$

 Then Write('Invalid String')

 Exit

4. [Next symbol must be blank]

$\text{NEXT} \leftarrow \text{NEXTCHAR}(\text{STRING})$

If $\text{NEXT} = ' '$

Then Write('VALID STRING')

Else Write('INVALID STRING')

Input String: a b c b a □

↑
NEXT

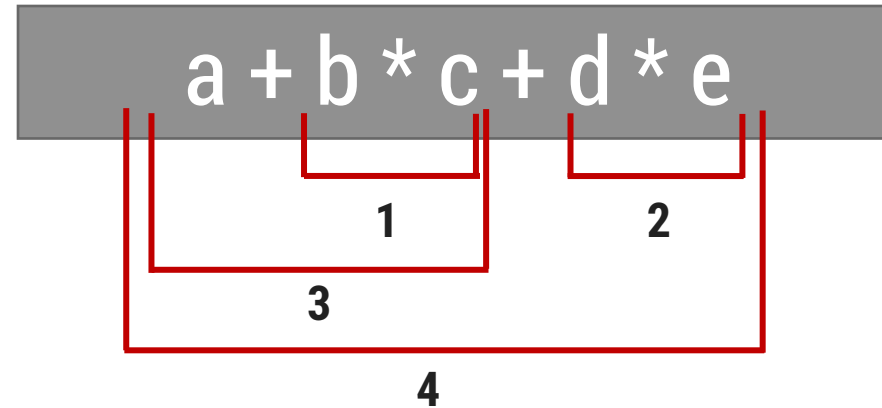
	Character Scanned	Stack Content
1 {		c
2 {	a	ca
	b	cab
	c	cab
3 {	b	ca
	a	c
	□	

Algorithm : RECOGNIZE

- ▶ Write an algorithm to determine if an input character string is of the form ***aⁱbⁱ*** where ***i ≥ 1***
- ▶ i.e. number of **a** ***should be equal*** to no of **b**

Polish Expression & their Compilation

► Evaluating Infix Expression



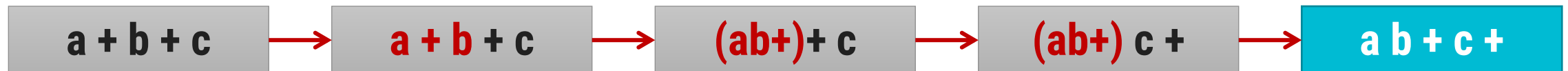
- A **repeated scanning** from left to right is needed as operators appear inside the operands.
- *Repeated scanning is avoided* if the infix expression is first **converted** to an equivalent parenthesis free **prefix or suffix (postfix) expression**.
- **Prefix Expression:** **Operator**, Operand, Operand
- **Postfix Expression:** Operand, Operand, **Operator**

Polish Notation

- ▶ This type of notation is known **Lukasiewicz Notation** or **Polish Notation** or **Reverse Polish Notation** due to Polish logician *Jan Lukasiewicz*.
- ▶ In both **prefix** and **postfix** equivalents of an infix expression, the ***variables are in same relative position***.
- ▶ The expressions in postfix or prefix form are ***parenthesis free*** and operators are rearranged according to rules of precedence for operators.

Polish Notation

Sr.	Infix	Postfix	Prefix
1	a	a	a
2	a + b	a b +	+ a b
3	a + b + c	a b + c +	+ + a b c
4	a + (b + c)	a b c + +	+ a + b c
5	a + (b * c)	a b c * +	+ a * b c
6	a * (b + c)	a b c + *	* a + b c
7	a * b * c	a b * c *	** a b c



Finding Rank of any Expression

$$E = (A + B * C / D - E + F / G / (H + I))$$

Note: $R = \text{Rank}$, Rank of Variable = 1, Rank of binary operators = -1

$$\begin{aligned} \text{Rank (E)} = & R(A) + R(+) + R(B) + R(*) + R(C) + R (/) + R(D) + R(-) + R(E) + R(+) + R(F) + R (/) + R(G) \\ & + R (/) + R(H) + R(+) + R(I) \end{aligned}$$

$$\text{Rank (E)} = 1 + (-1) + 1 + (-1) + 1 + (-1) + 1 + (-1) + 1 + (-1) + 1 + (-1) + 1 + (-1) + 1 + (-1) + 1$$

$$\text{Rank (E)} = 1$$

Any Expression is valid if Rank of that expression is 1

Convert Infix to Postfix Expression

Symbol	precedence
+, -	1
*, /	2
^	3

Algorithm : REVPOL

- ▶ Given an input string **INFIX** containing an infix expression which has been padded on the right with ')'.
 - ▶ This algorithm *converts INFIX into reverse polish* and places the result in the string **POLISH**.
 - ▶ All symbols have precedence value given by the table.
 - ▶ Stack is represented by a vector **S**, **TOP** denotes the top of the stack, algorithm **PUSH** and **POP** are used for stack manipulation.
 - ▶ Function **NEXTCHAR** returns the next symbol in given input string.
 - ▶ The integer variable **RANK** contains the rank of expression.
 - ▶ The string variable **TEMP** is used for temporary storage purpose.

Infix to post fix

1. Scan the infix expression from left to right.
2. If the scanned character is an operand, append it to the post-fix expression.
3. If the scanned character is an '(', push it into the stack.
4. If the scanned character is an ')', then Pop all the contents of the stack until the respective left parenthesis is popped and append each popped symbol to Postfix Expression.
5. If the scanned character is an operator (+, -, *, /),
 - If the precedence of the scanned operator is greater than the precedence of the operator in the stack or the stack is empty or the stack contains a '(', push the character into the stack.
 - Else, Pop all the operators from the stack that are greater than or equal to in precedence than that of the scanned operator. After doing that Push the scanned operator to the stack.
6. Repeat steps 2-5 until the entire infix expression is scanned.

Precedence order and Associativity order

Operators	Symbols
Parenthesis	{ }, (), []
Exponential notation	^
Multiplication and Division	*, /
Addition and Subtraction	+, -

Operators	Associativity
^	Right to Left
*, /	Left to Right
+, -	Left to Right

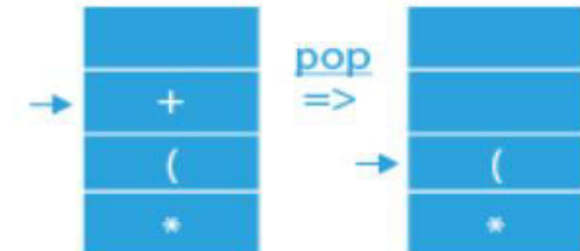
► Infix expression: $a * (b + c + d)$

(7) Infix
 $a * (b + c + d)$



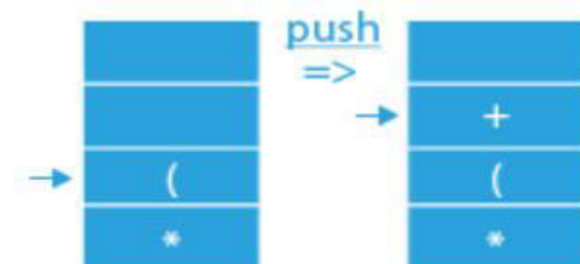
=> '+' is an operator, so push it into stack after removing higher or equal priority operators from top of stack.

=> Current top of stack is '+', it has equal precedence with input symbol '+'. So, pop it and append to postfix.

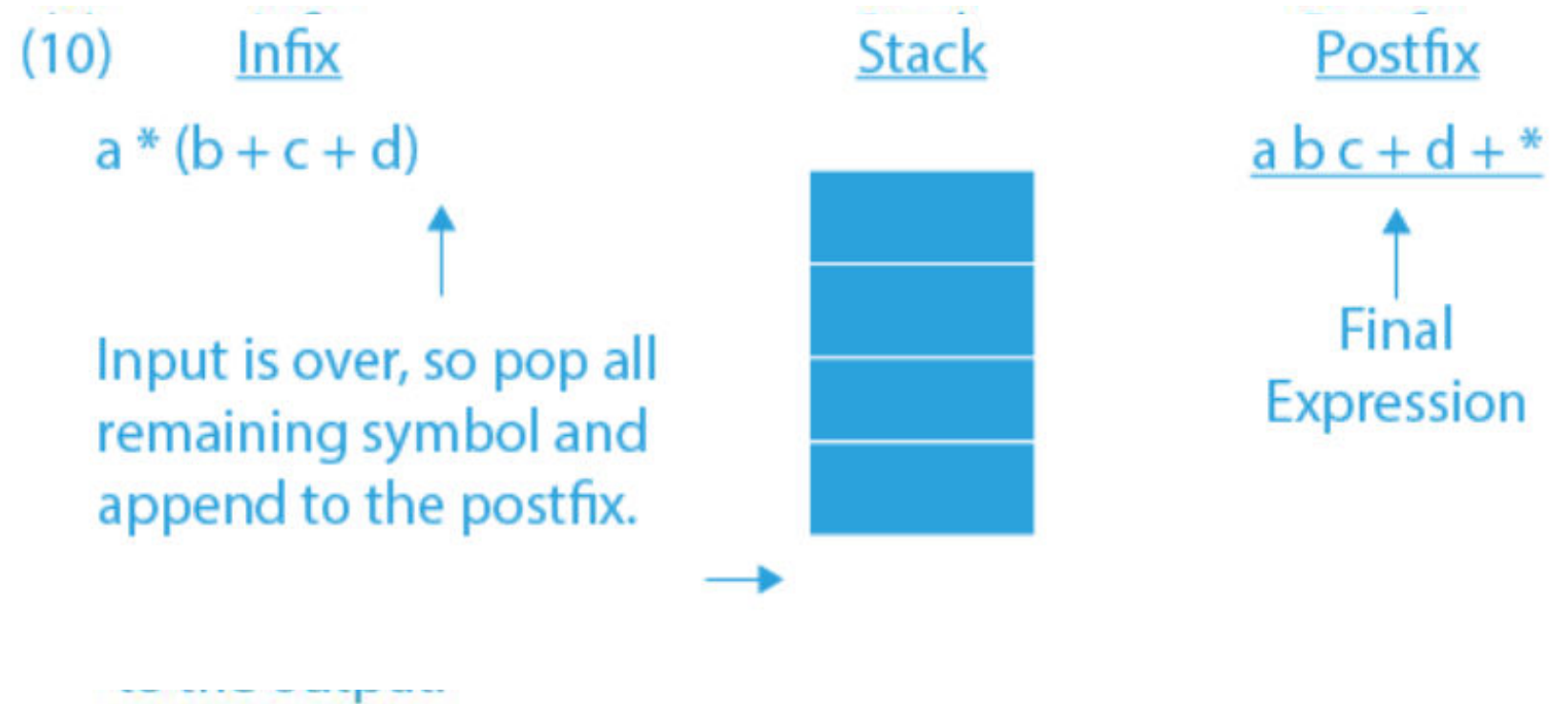


postfix => $a \ b \ c \ +$

=> Now the top of stack is '(', so push the input operator.



► Infix expression: $a * (b + c + d)$



Example

$$(a + (b * c) / (d - e)) = abc^*de-/+$$

SYMBOL	OPSTACK	POSTFIX
(	(	EMPTY
a	(	a
+	(+	a
(	(+(	a
b	(+(	ab
*	(+(*	ab
c	(+(*	abc
)	(+	abc*
/	(+(/	abc*
(	(+/(	abc*
d	(+/(	abc*d
-	(+/(-	abc*d
e	(+/(-	abc*de
)	(+(/	abc*de-
)	Empty	abc*de-/+

Perform following operations

► **Convert** the following **infix** expression to **postfix**. Show the stack contents.

↳ $A\$B-C*D+E\F/G

↳ $A + B - C * D * E \$ F \$ G$

↳ $a+b*c-d/e*h$

↳ $((a+b^c^d)*(e+f/d))$

► Convert the following **infix** expression to **prefix**.

↳ $A + B - C * D * E \$ F \$ G$

General Infix-to-Postfix Conversion

Create an empty stack called **stack** for keeping operators. Create an empty list for output.

Read the character list from left to right and perform following steps

- 1 If the **character** is an **operand (Variable)**, **append** it to the **end** of the **output** list
- 2 If the **character** is a **left parenthesis '('**, **push** it on the **stack**
- 3 If the **character** is a **right parenthesis ')'**, **pop** the **stack** until the corresponding **left parenthesis '('** is removed. **Append** each **operator** to the **end of the output** list.
- 4 If the token is an **operator**, *****, **/**, **+**, or **-**, **push** it on the **stack**. However, **first remove any operators** already on the **stack** that **have higher or equal precedence** and **append them to the output** list.

$$(a + b ^ c ^ d) * (e + f / d)$$

Evaluation of postfix expression

- ▶ Each **operator** in **postfix** string refers to the *previous two operands* in the string.
- ▶ Each time we **read** an **operand**, we **PUSH** it onto **Stack**.
- ▶ When we reach an **operator**, its **operands** will be **top two elements** on the stack.
- ▶ We can then **POP** these two elements, perform the indicated operation on them and **PUSH** the result on the stack so that it will be available for use as an operand for the next operator.

Evaluation of postfix expression

Evaluate Expression: 5 6 2 - +

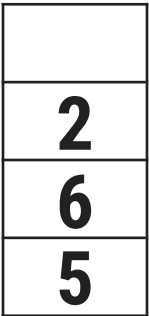
Empty Stack



Read 5, is it operand? PUSH

Read 6, is it operand? PUSH

Read 2, is it operand? PUSH



Read -, is it operator? POP
two symbols and perform
operation and PUSH result



Operand 1 - Operand 2



Answer

Read next symbol, if it
is end of string, POP
answer from Stack



Read +, is it operator? POP
two symbols and perform
operation and PUSH result



Operand 1 + Operand 2

Algorithm: EVALUATE_POSTFIX

- ▶ Given an input string **POSTFIX** representing postfix expression.
- ▶ This algorithm evaluates postfix expression and put the result into variable **VALUE**.
- ▶ Stack is represented by a vector **S**, **TOP** denotes the top of the stack, Algorithm **PUSH** and **POP** are used for stack manipulation.
- ▶ Function **NEXTCHAR** returns the next symbol in given input string.
- ▶ **OPERAND1**, **OPERAND2** and **TEMP** are used for temporary variables
- ▶ **PERFORM_OPERATION** is a function which performs required operation on **OPERAND1** & **OPERAND2**.

Algorithm: EVALUATE_POSTFIX

1. [Initialize Stack]

TOP $\leftarrow$ 0

VALUE $\leftarrow$ 0

2. [Evaluate the postfix expression]

Repeat until last character

TEMP $\leftarrow$ NEXTCHAR (POSTFIX)

If TEMP is DIGIT

Then PUSH (S, TOP, TEMP)

Else OPERAND2 $\leftarrow$ POP (S, TOP)

OPERAND1 $\leftarrow$ POP (S, TOP)

VALUE $\leftarrow$ PERFORM_OPERATION(OPERAND1, OPERAND2, TEMP)

PUSH (S, TOP, VALUE)

3. [Return answer from stack]

Return (POP (S, TOP))

Evaluation of postfix expression

Evaluate Expression: 5 4 6 + * 4 9 3 / + *

Evaluate Expression: 7 5 2 + * 4 1 1 + / -

Evaluate Expression: 12, 7, 3, -, /, 2, 1, 5, +, *, +

Infix to Prefix

1. Scan the valid infix expression from **right to left**.
2. Initialize the empty character **stack** and empty **prefix string**.
3. If the scanned character is an **operand**, **append** it to the prefix string.
4. If the scanned character is **closing parenthesis**, **push** it to the stack.
5. If the scanned character is **opening parenthesis**, **pop** all the characters of the stack and concatenate to the end of the prefix string until closing parenthesis occurs. Pop the closing parenthesis.
6. If the scanned character is an Operator and the stack is not empty, compare the precedence of the character with the element on top of the stack.
 - If top element of the Stack has higher precedence over the scanned character, pop the stack
 - else push the scanned character to stack. Repeat this step until the stack is not empty and top Stack has precedence over the character.
7. Repeat steps 3 to 6 until all characters are scanned.

$$(a + (b * c) / (d - e)) = +a/*bc-de$$

NOTE: scan the infix string in reverse order.

SYMBOL	PREFIX	OPSTACK
)	Empty	)
)	Empty	))
e	e	))
-	e	))-
d	de	))-
(	-de	)
/	-de	)/
)	-de	)/)
c	c-de	)/)
*	c-de	)/)*
b	bc-de	)/)*
(	*bc-de	)/
+	/*bc-de	)+
a	a/*bc-de	)+
(	+a/*bc-de	Empty

Algorithm: EVALUATE_PREFIX

- ▶ Given an input string **PREFIX** representing prefix expression.
- ▶ This algorithm evaluates prefix expression and put the result into variable **VALUE**.
- ▶ Stack is represented by a vector **S**, **TOP** denotes the top of the stack, Algorithm **PUSH** and **POP** are used for stack manipulation.
- ▶ Function **NEXTCHAR** returns the next symbol in given input string.
- ▶ **OPERAND1**, **OPERAND2** and **TEMP** are used for temporary variables
- ▶ **PERFORM_OPERATION** is a function which performs required operation on **OPERAND1** & **OPERAND2**.

Algorithm: EVALUATE_PREFIX

1. [Initialize Stack]

TOP $\leftarrow$ 0

VALUE $\leftarrow$ 0

2. [Evaluate the prefix expression]

Repeat from last character up to first

TEMP $\leftarrow$ NEXTCHAR (PREFIX)

If TEMP is DIGIT

Then PUSH (S, TOP, TEMP)

Else OPERAND1 $\leftarrow$ POP (S, TOP)

OPERAND2 $\leftarrow$ POP (S, TOP)

VALUE $\leftarrow$ PERFORM_OPERATION(OPERAND1, OPERAND2, TEMP)

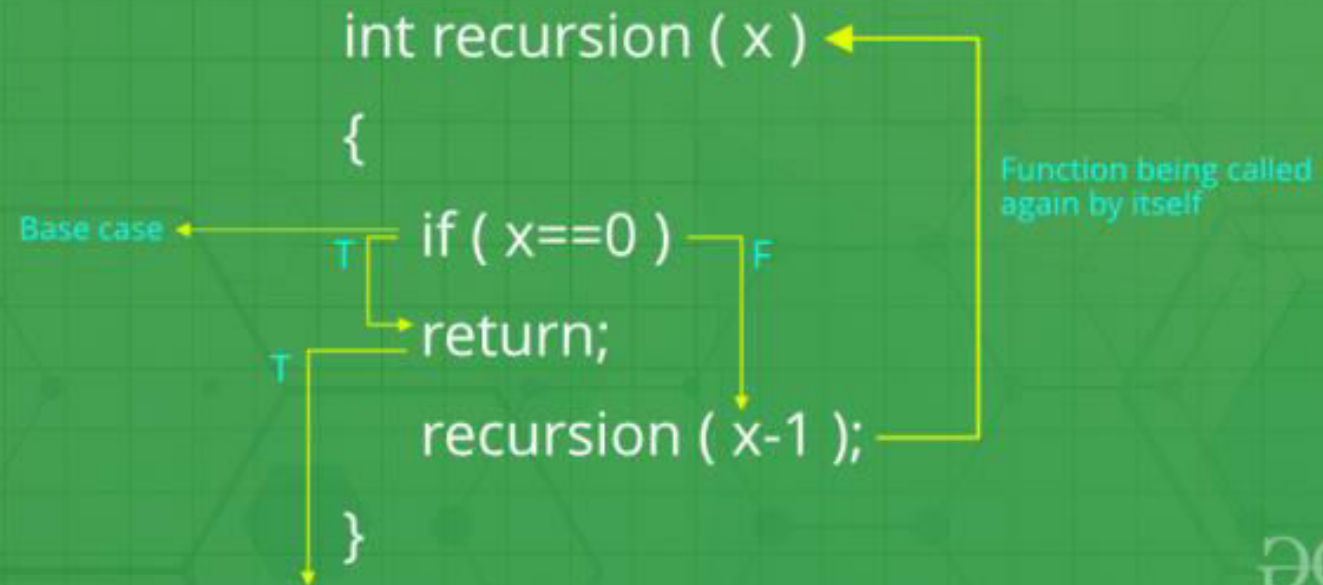
PUSH (S, TOP, VALUE)

3. [Return answer from stack]

Return (POP (S, TOP))

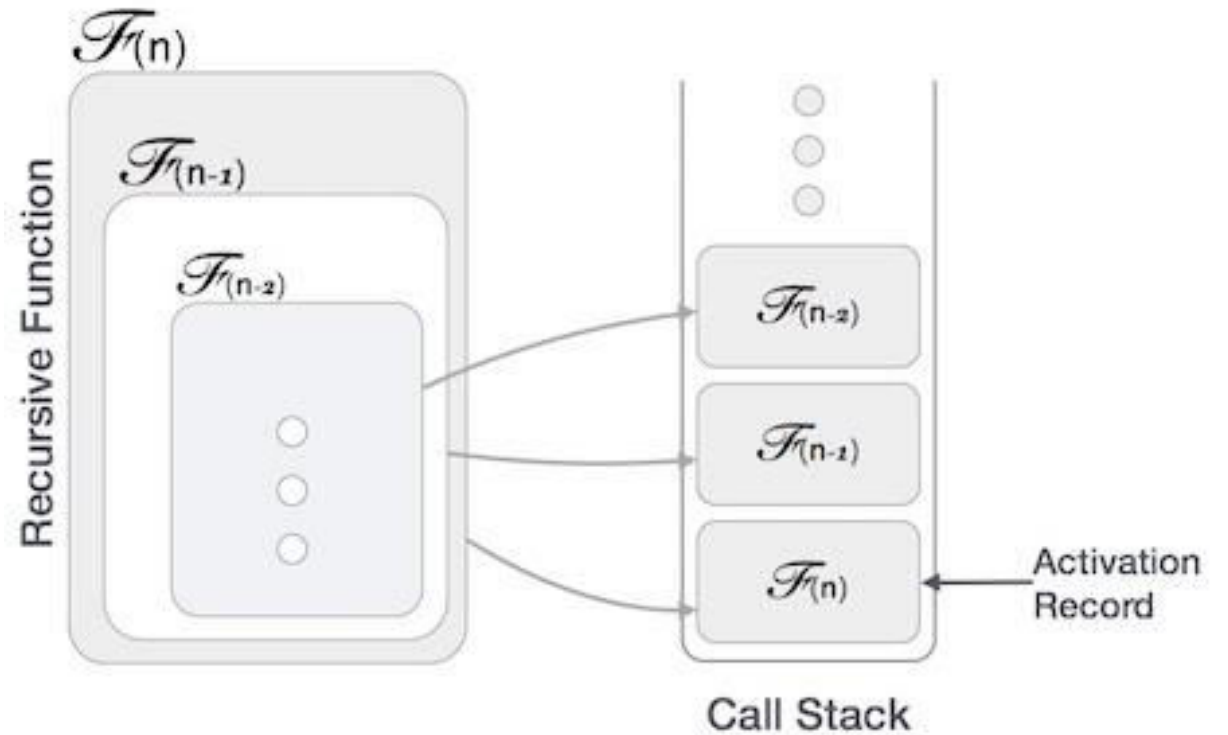
Recursion

Recursive Functions



Example:

```
int fun(int x)
{
    if(x < 1)
        return;
    fun(x - 1);
    printf("%d ", x);
}
```



Example:

```
int fun(int n)
{
    if (n <= 1) // base case
        return 1;
    else
        return n + fun(n-1);
}
```

```
int fun(int n)
{
    //wrong base case (it may cause stack overflow).
    if (n == 100)
        return 1;
    else
        return n + fun(n-1);
}
```

Algorithm to find factorial using recursion

Factorial of n

- ▶ Factorial of any number n is denoted as n! and is equal to
- ▶ $n! = 1 \times 2 \times 3 \times \dots \times (n - 2) \times (n - 1) \times n$
- ▶ Factorial of 3:

$$\begin{aligned} 3! &= 1 \times 2 \times 3 \\ &= 6 \end{aligned}$$

```
int factorial(int n)
{
    int fact = 1, i;

    for (i = 2; i <= n; i++)
        fact = fact * i;

    return fact;
}
```

Algorithm: FACTORIAL

Step 1: Start

Step 2: Read number n

Step 3: Call `factorial(n)`

Step 4: Print factorial f

Step 5: Stop

factorial(n)

Step 1: If $n==1$ then return 1

Step 2: Else

$f=n*\text{factorial}(n-1)$

Step 3: return f

FACTORIAL

```
int fun(int n)
{
    if (n < = 1) // base case
        return 1;
    else
        return n * fun(n-1);
}
```

Examples:

```
#include<stdio.h>
void fun(int x)
{
    if(x > 0)
    {
        fun(--x);
        printf("%d\t", x);
        fun(--x);
    }
}

int main()
{
    int a = 4;
    fun(a);
    getchar();
    return 0;
}
```

Output: 0 1 2 0 3 0 1

Examples

a.
On recursion, the value of `f(513,2)`

```
int f(int n, int r)
{
    if(n<=0)
        return 0;
    else
        return ( n%r + f(n/r, r) )
}
```

b.

```
void myFun(unsigned int n)
{
    if(n != 0)
    {
        myFun(n/2);
        printf("%d", n % 2);
    }
}

int main()
{
    myFun(25);
    getchar();
    return 0;
}
```